Introduzione

Questo progetto nasce per offrire un supporto **interattivo e strutturato** a chi si avvicina al linguaggio **Prolog**. L'applicazione desktop non si limita a presentare contenuti, ma guida l'utente in un vero e proprio **percorso didattico progressivo**: ogni livello rappresenta un tassello fondamentale che introduce nuovi concetti della programmazione logica, bilanciando in modo efficace teoria ed esercitazioni pratiche.

Per consolidare l'apprendimento, il sistema propone **quiz a risposta multipla** e attività di **completamento (fill-in-the-blank)**.

Funzionalità principali

- **Accesso riservato:** gestione del profilo tramite autenticazione utente.
- **Curriculum strutturato:** percorso di apprendimento organizzato per livelli di difficoltà.
- **Didattica interattiva:** quiz mirati con correzione automatica e istantanea.
- **Dashboard dei progressi:** monitoraggio continuo del percorso con statistiche su livelli completati, punteggi e percentuale di avanzamento.

Processo di sviluppo →

Processo di Sviluppo

Ruoli e Collaborazione

- **Product Owner – Gianfranco Branco** Si è occupato di definire la visione strategica e l'architettura del sistema, assicurando che lo sviluppo dei requisiti (utenti e livelli) rispondesse agli obiettivi di progetto. Ha coordinato le priorità del backlog e gestito la documentazione tecnica, partecipando attivamente alla parte implementativa e alla persistenza dati.
- **Scrum Master – Francesco Agnoletti** Ha facilitato l'organizzazione dei cicli di sviluppo e rimosso gli ostacoli tecnici, garantendo la fluidità del workflow e la qualità del codice. Si è focalizzato sulla logica d'interazione, curando lo sviluppo dei quiz e delle interfacce grafiche, contribuendo in modo operativo a tutte le fasi del progetto.

Metodologia SCRUM-inspired

Il ciclo di vita del software è stato scandito da sprint della durata di 10 giorni, garantendo flessibilità e adattamento ai requisiti.

Meeting

- **Initial Planning**: Definizione dell'architettura e degli obiettivi a lungo termine.
- **Sprint Planning**: Selezione dei task, cercando di bilanciare i carichi di lavoro.
- **Daily Scrum**: Allineamenti rapidi (15 min) per sincronizzare lo sviluppo e identificare ostacoli.
- **Review & Retrospective**: Analisi del software prodotto e miglioramento delle pratiche di lavoro al termine di ogni sprint.

Gestione del Progetto

Per la gestione dei task del **Product Backlog** sono gestiti come segue:

1. **Task Critici**: Assegnazione diretta durante il planning.
2. **Task Self-Service**: Scelta autonoma dai membri del team una volta completati gli incarichi primari.

Versioning e Convenzioni

Il codice sorgente è gestito tramite **Git**, adottando un approccio basato su branch dedicati per funzionalità:

- **main**: branch dedicato esclusivamente alle release stabili.
- **branch di feature**: questi vengono creati ad hoc per ogni nuova funzionalità o modifica e successivamente integrati in `main` tramite merge.

Definition of Done (DoD)

- **Superamento Test Automatizzati**: Il codice deve superare con successo la suite di test unitari (ScalaTest) dedicata ai componenti core, assicurando la stabilità delle logiche di business prima del rilascio in main.
- **Verifica della Compilazione Locale**: Il codice deve compilare correttamente nell'ambiente di sviluppo di entrambi i membri del team, garantendo l'assenza di conflitti di dipendenze o errori di sintassi prima del commit.
- **Documentazione Scaladoc**: I componenti core devono essere documentati tramite Scaladoc.
- **Codice Auto-esplicativo**: Il codice deve rispettare gli standard di Self-Documenting Code, utilizzando una nomenclatura chiara per variabili e metodi per minimizzare commenti ridondanti.
- **Standard Stilistici**: La formattazione del codice deve essere coerente con i parametri del progetto (Scalafmt) per garantire la leggibilità collettiva del repository.

Strategia di Qualità

Test Driven Development (TDD)

Abbiamo applicato il paradigma TDD sui componenti core (Model e Controller):

1. **Red**: Scrittura del test fallimentare (es. validazione login).
2. **Green**: Implementazione minima per superare il test.
3. **Refactor**: Ottimizzazione del codice mantenendo i test superati.

Code Quality

Per l'uniformità del codice sorgente è stato adottato **Scalafmt**, uno strumento di formattazione automatica che assicura il rispetto degli standard stilistici condivisi dal team, migliorando la leggibilità e la manutenibilità del repository.

Building

Per la gestione del ciclo di vita del software è stato adottato Sbt, utilizzato come strumento principale per la risoluzione automatica delle dipendenze, la compilazione del codice sorgente e l'esecuzione della suite di test.

Report

Il report è stato realizzato con **Jekyll** e il tema Just the Docs, sfruttando l'integrazione nativa con GitHub Pages. I contenuti, redatti in Markdown, vengono poi versionati su Git.

Analisi dei Requisiti →

Analisi dei Requisiti

Di seguito viene definita la **vision del progetto** e l’insieme dei **requisiti** che l’applicazione dovrà soddisfare.

Business

Si vuole sviluppare un’applicazione desktop a supporto dell’apprendimento del linguaggio **Prolog**, rivolta a utenti che si avvicinano per la prima volta alla programmazione logica.

L’obiettivo principale è fornire un **percorso didattico guidato e progressivo**, strutturato per livelli che consenta all’utente di acquisire le nozioni fondamentali del linguaggio.

Le principali finalità includono:

- **Apprendimento guidato**: facilitare la comprensione dei concetti chiave di Prolog (fatti, regole, query, ricorsione, backtracking).
- **Verifica delle conoscenze**: consentire il consolidamento dell’apprendimento tramite quiz interattivi con valutazione automatica.
- **Motivazione e progressione**: incentivare la continuità nello studio attraverso il completamento dei livelli e il tracciamento dei progressi.
- **Accessibilità didattica**: offrire uno strumento semplice e intuitivo, utilizzabile anche da utenti senza esperienza pregressa di programmazione logica.

Modello di dominio

Gli elementi principali del sistema sono:

- **User**: rappresenta l’utilizzatore dell’applicazione, dove sarà possibile gestirne l'identità e lo storico dei progressi di apprendimento.
- **Level**: rappresenta un livello del percorso didattico, dedicato a specifici concetti del linguaggio Prolog.
- **Theory**: contenuto testuale che introduce la teoria per svolgere il livello.
- **Question**: singola domanda di un quiz, a risposta multipla o di completamento.
- **Answer**: risposta fornita dall’utente a una domanda.
- **Progress**: stato di avanzamento dell’utente, comprensivo di livelli completati, punteggio e percentuale di completamento.

Requisiti funzionali

Utente

- L’utente deve poter accedere all’applicazione tramite **autenticazione**.
- L’utente deve poter registrarsi
- L’utente deve poter editare il proprio Nome
- L’utente deve poter eliminare la propria utenza
- L’utente deve poter visualizzare un **percorso didattico strutturato per livelli**.
- L’utente deve poter svolgere esercizi di verifica sotto forma di:
  - quiz a risposta multipla;
  - quiz di completamento (*fill-in-the-blank*).
- L’utente deve ricevere un **feedback immediato** sull’esito delle risposte.
- L’utente deve poter visualizzare i propri **progressi**, inclusi:
  - Conteggio livelli completati
  - Percentuale di risposte corrette
  - Conteggio quiz tentati
  - Conteggio ore di utilizzo.

Sistema

- Il sistema deve gestire l’autenticazione e la memorizzazione dei profili utente.
- Il sistema deve memorizzare e presentare all’utente i livelli ed i quiz disponibili.
- Il sistema deve valutare automaticamente le risposte fornite nei quiz.
- Il sistema deve aggiornare e memorizzare lo stato di avanzamento dell’utente.
- Il sistema deve fornire una **interfaccia grafica** per la fruizione dei contenuti e dei quiz.

Requisiti non funzionali

- **Usabilità**: l’interfaccia grafica deve essere intuitiva e facilmente utilizzabile
- **Estendibilità**: il sistema deve gestire l’aggiunta di nuovi livelli e quiz
- **Testabilità**: devono essere presenti test per verificare il corretto funzionamento delle funzionalità principali
- **Portabilità**: l’applicazione deve poter essere eseguita su diversi sistemi operativi desktop
- **Performance**: il sistema deve garantire tempi di risposta rapidi durante la navigazione e la valutazione dei quiz
- **Manutenibilità**: il codice deve essere strutturato in modo modulare e facilmente manutenibile

Implementazione

- **Linguaggio**: Scala 3.8.1
- **GUI**: Scala FX 23.0.1-R34
- **Testing**: ScalaTest 3.2.19
- **Build Tool**: SBT 1.12.1
- **Java**: 21.0.10
- **uPickle**: 3.1.0

Requisiti opzionali

- **Playground Prolog** per la scrittura ed esecuzione di query.
- **Modalità di gioco “sopravvivenza”**, in cui l’utente risponde a una sequenza casuale di domande che termina al primo errore.

Design Architetturale →

Design Architetturale

Pattern Architetturale

L'architettura di Prolog for Dummies è stata progettata seguendo i principi della programmazione funzionale e reattiva, garantendo un sistema robusto e privo di effetti collaterali. L'approccio funzionale si concretizza nell'adozione sistematica dell'immutabilità per l'intero dominio del modello: entità come utenti, livelli e quesiti sono gestite tramite case class di Scala 3, assicurando che ogni variazione di stato produca nuove istanze anziché modificare quelle esistenti. Parallelamente, la natura reattiva del sistema permette all'interfaccia utente in ScalaFX di operare secondo un paradigma event-driven, in cui gli input dell'utente attivano flussi di elaborazione asincroni gestiti dai controller.

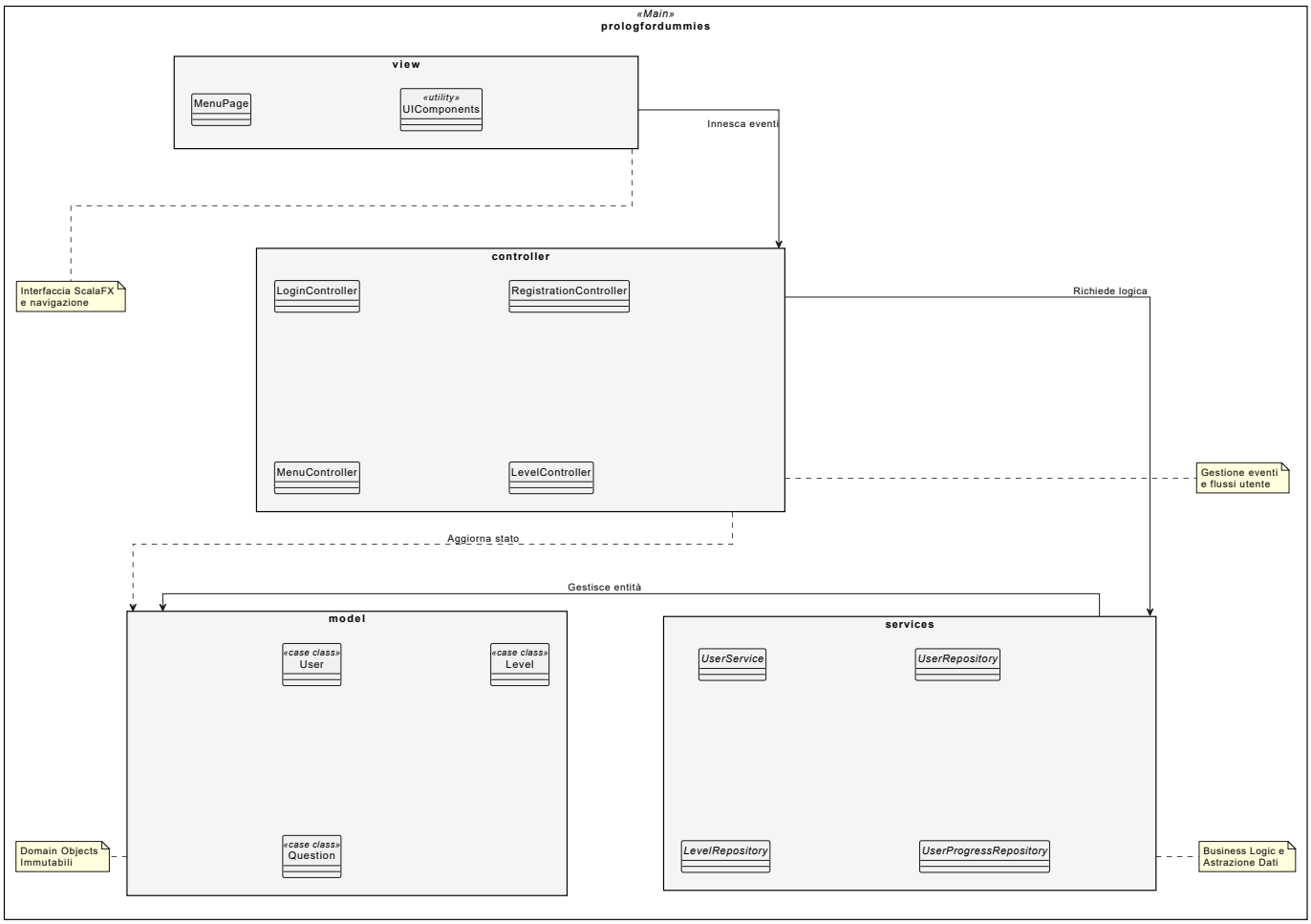
Architettura del Progetto

Il sistema adotta un pattern Model-View-Controller (MVC) esteso con uno strato di Service e Repository per ottimizzare la separazione delle responsabilità (Separation of Concerns).

Repository Pattern: Isola la logica di accesso ai dati (JSON), rendendo il resto dell'applicazione indipendente dai dettagli della persistenza.

Service Layer: Centralizza la Business Logic, fungendo da mediatore tra i Controller e i Repository per garantire riusabilità e facilità di testing.

Questa struttura a strati permette di evolvere il sistema senza impattare sull'interfaccia utente o sul flusso di controllo.



Model

Composto da case class immutabili (User, Level, Question). L'immutabilità è una scelta architetturale cruciale: ogni variazione di stato (es. una risposta corretta) non modifica l'oggetto esistente ma produce una nuova versione del modello, garantendo thread-safety e facilità di testing.

View

Sviluppata in ScalaFX, adotta un approccio gerarchico e dichiarativo. La navigazione è centralizzata, garantendo una transizione fluida tra i diversi stati dell'applicazione. I componenti dell'interfaccia sono stati progettati per essere altamente modulari e riutilizzabili (grazie alla classe UIComponents), permettendo di mantenere coerenza grafica e facilità di estensione.

Controller

Agisce come orchestratore. Riceve gli input dalla View e interagisce con i Service. Non contiene logica di business complessa, ma delega la validazione e il salvataggio agli strati inferiori.

Service & Repository Pattern

Introdotto per astrarre la persistenza. Il UserService coordina le operazioni di business, mentre le interfacce Repository isolano l'applicazione dal formato di archiviazione fisico (JSON). Questo permette, in futuro, di sostituire il database senza modificare troppo la logica dei controller.

Componenti del Sistema e Logica di Valutazione

Sebbene l'applicazione sia attualmente standalone, l'architettura è predisposta per una futura evoluzione distribuita. Un componente chiave è il Motore di Valutazione Prolog:

Integrazione tuProlog: Il sistema incapsula una libreria di interpretazione Prolog. Questa scelta permette di separare il "cosa" viene insegnato dal "come" viene gestito il quiz.

Validazione Dinamica: A differenza di un quiz a risposta multipla statico, il sistema può interrogare una base di conoscenza reale per verificare la correttezza delle query scritte dall'utente.

Scelte Tecnologiche Cruciali

Le decisioni tecniche che hanno plasmato l'architettura includono:

Scala 3: L'utilizzo delle given e dei parametri contestuali permette di implementare un pattern di Dependency Injection nativo. Questo riduce drasticamente l'accoppiamento tra i Controller e le implementazioni concrete dei Repository, facilitando lo switch tra diverse strategie di persistenza (ad esempio, passando da un'archiviazione su file a una in-memory) senza modificare la logica dei client.

JSON (uPickle): Scelto come formato di persistenza per la sua leggibilità e facilità di debug. L'uso del formato JSON permette una gestione dei dati "file-based", eliminando la necessità di configurare database tradizionali e garantendo la massima portabilità dell'applicativo (Zero-Config Deployment).

Pattern Either per l'Error Handling: Al posto delle eccezioni, i servizi restituiscono tipi Either[String, A]. Questa scelta architetturale impone una gestione esplicita dei fallimenti nei controller, rendendo l'applicazione più robusta e priva di crash imprevisti.

Design di Dettaglio →

Design di Dettaglio

Questo documento descrive il funzionamento dettagliato dei principali componenti del sistema **Prolog for Dummies**, evidenziando responsabilità, interazioni e flussi applicativi tra i vari strati dell'architettura.

Il progetto è organizzato secondo un approccio **MVC esteso** con layer aggiuntivi di **Service** e **Repository**, così da garantire separazione delle responsabilità, manutenzione semplice ed estendibilità.

Vista generale dei componenti

Il sistema è composto da:

- **Model**: rappresenta il dominio applicativo e i dati persistenti/in memoria;
- **View**: gestisce l'interfaccia grafica e l'interazione con l'utente;
- **Controller**: coordina il flusso tra interfaccia e logica applicativa;
- **Service**: gestisce la comunicazione tra i componenti necessari per la logica applicativa;
- **Repository**: gestisce l'accesso ai dati su file JSON;
- **Motore Prolog**: valuta le risposte degli esercizi basati su Prolog.

Design del Model

Il livello di dominio è stato progettato in modo da privilegiare **immutabilità**, **coerenza dei dati** e **riuso della logica di calcolo**.

Entità principali

User

Rappresenta l'utente dell'applicazione. Contiene le informazioni necessarie per autenticazione ed il salvataggio dei progressi.

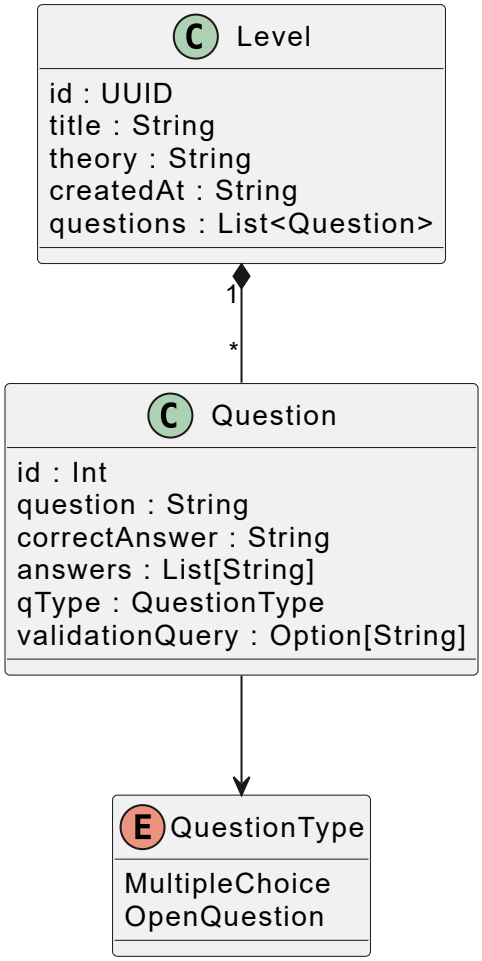
Level

Rappresenta un livello del percorso didattico. Ogni livello contiene:

- contenuto teorico;
- una sequenza di domande a risposta multipla o aperte;
- metadati utili alla navigazione e alla valutazione.

Dettaglio del livello

Struttura dati levels.json



Question

Rappresenta una singola domanda del quiz. Il modello supporta tipologie differenti, in particolare:

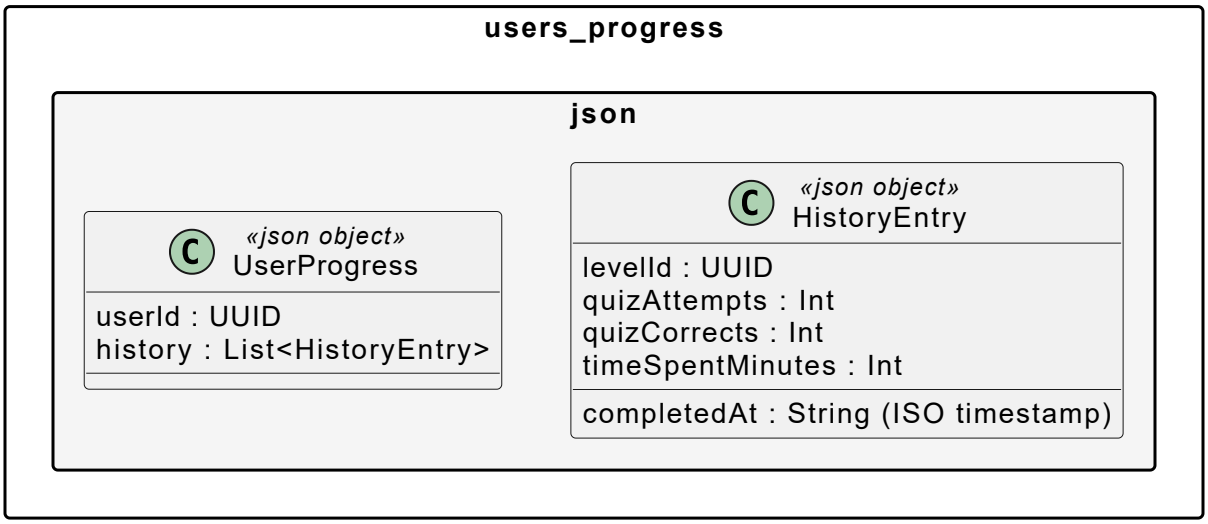
- **MultipleChoice**
- **OpenQuestion / Fill-in-the-blank**

UserProgress

Aggrega la cronologia dei progressi utente, registrando per ogni livello completato i relativi KPI e dati di sessione.

Dettaglio di UserProgress

Schema dati - users_progress.json



LevelRecord

Modella il completamento di un singolo livello, includendo i dati raccolti durante la sessione di gioco.

Principi di progettazione applicati

Immutabilità

Le entità del dominio sono state progettate come strutture immutabili, così da evitare modifiche accidentali e rendere il flusso dei dati più prevedibile.

Rich Domain Model

Alcuni calcoli, come:

- ore totali di utilizzo,
- numero di risposte corrette,
- accuratezza globale,

sono incapsulati direttamente nel modello, riducendo la complessità dei controller.

State Management

La gestione della sessione di livello si basa su uno stato esplicito, distinguendo tra:

- **Idle**: nessun livello in corso;
- **ActiveLevel**: livello attivo con contatori e timestamp.

Questo approccio evita incoerenze tra stato temporaneo e dati persistenti.

Design della persistenza

La persistenza è stata progettata con un approccio **file-based JSON**, per semplificare il deployment e mantenere l'applicazione portabile.

Repository Pattern

Ogni entità principale possiede:

- una trait repository;
- una implementazione concreta `Impl` .

I repository astraggono l'accesso ai dati e permettono di modificare facilmente la tecnologia di persistenza in futuro.

Entità persistite

Le informazioni salvate su file riguardano principalmente:

- utenti;
- progressi utente;
- livelli.

Strategia di aggiornamento

Per i progressi utente viene adottato un approccio **read-modify-write**, che:

1. legge lo stato corrente dal file;
2. aggiorna solo la porzione necessaria;
3. riscrive il contenuto in modo coerente.

Questa soluzione riduce il rischio di perdita dati e mantiene il formato semplice da gestire.

Design del layer Service

Il layer di servizio ha il compito di centralizzare la **business logic** e di fare da intermediario tra controller e repository.

UserService

Si occupa delle operazioni sugli utenti, tra cui:

- registrazione;
- validazione dei dati;
- gestione degli errori;
- controllo dei duplicati.

L'uso di `Either[String, A]` consente di rappresentare in modo esplicito i fallimenti senza ricorrere a eccezioni come meccanismo ordinario di controllo del flusso.

PrologGrader

È il componente responsabile della valutazione delle risposte basate su Prolog.

In particolare:

- verifica la correttezza sintattica;
- unisce conoscenza del livello e soluzione dell'utente;
- esegue la query di test;
- determina l'esito della risposta.

Questo permette di distinguere tra:

- correttezza testuale;
- correttezza logico-sintattica.

Design dei Controller

I controller hanno la funzione di coordinare il flusso tra interfaccia utente e servizi, senza contenere logica di business complessa.

Responsabilità generali

Ogni controller:

- riceve gli eventi dalla View;
- valida gli input minimi;
- invoca i service o i repository;
- aggiorna la navigazione;
- gestisce eventuali messaggi di errore o feedback.

Iniezione delle dipendenze

La progettazione sfrutta i meccanismi di Scala 3 con:

- `given`
- parametri contestuali `using`

per ottenere una forma di dependency injection nativa e poco invasiva.

Controller principali

I controller coprono le funzionalità principali del sistema:

- autenticazione;
- registrazione;
- modifica utente;
- navigazione menu;
- selezione dei livelli;
- gestione del quiz;
- visualizzazione statistiche.

Design della View

La View è stata realizzata con **ScalaFX**, con un approccio dichiarativo e modulare.

Principi adottati

Coerenza visiva

L'interfaccia mantiene uno stile uniforme grazie alla centralizzazione di componenti riutilizzabili.

Riutilizzo

Sono stati definiti componenti comuni per:

- pulsanti;
- layout base;
- modali personalizzate;
- navigazione tra schermate.

Separazione delle responsabilità

La View si occupa solo della presentazione e dell'interazione, delegando la logica applicativa ai controller.

Schermate principali

Le principali schermate dell'applicazione includono:

- splash screen;
- login;
- registrazione;
- modifica profilo;
- menu principale;
- elenco livelli;
- quiz(a risposta multipla o aperta);
- statistiche utente.

Dialoghi e feedback

Le finestre di conferma personalizzate sono state progettate per garantire coerenza grafica ed un controllo maggiore dell'esperienza utente.

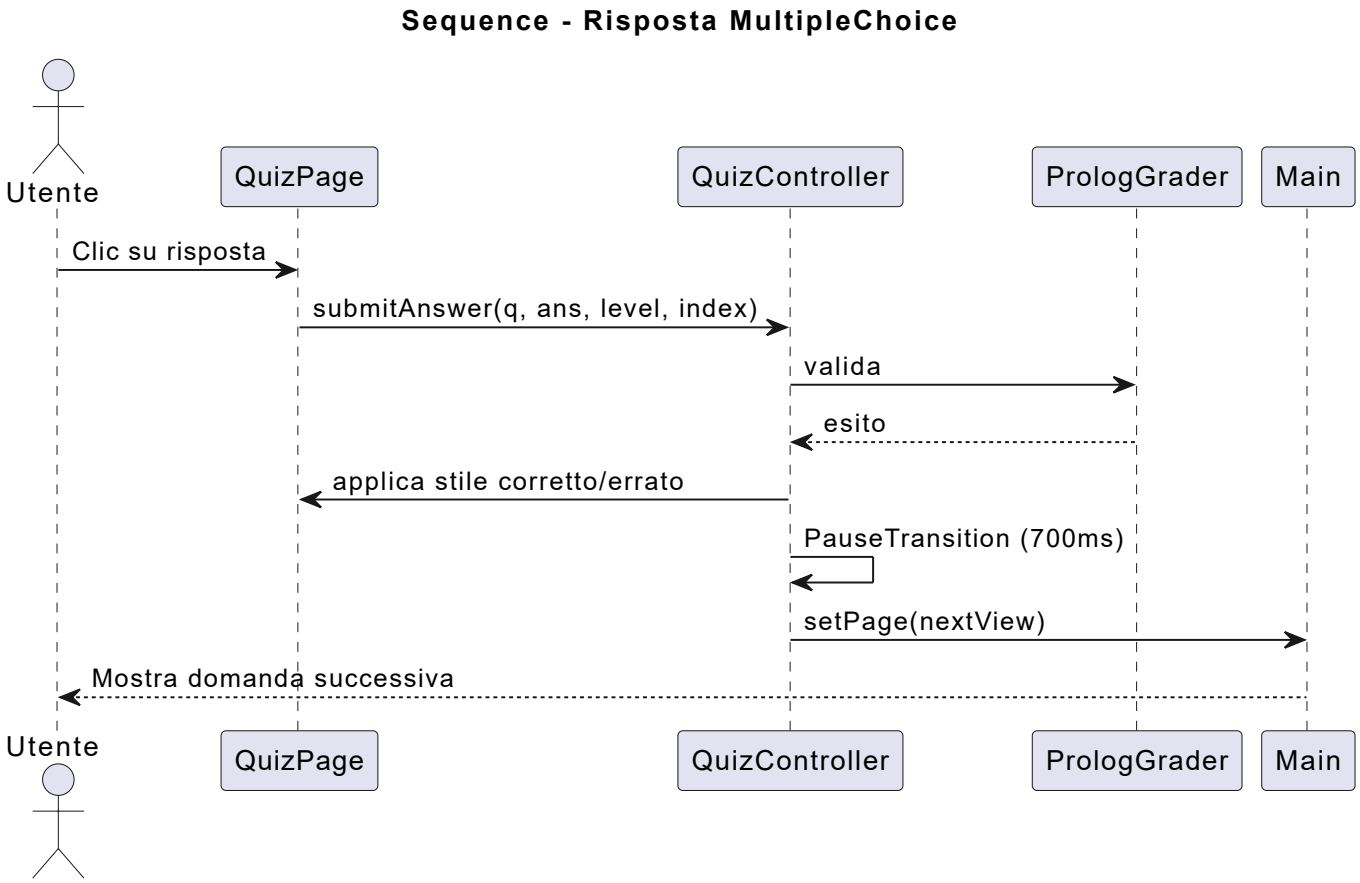
Design del flusso di navigazione

Il flusso applicativo segue una sequenza chiara:

1. avvio dell'applicazione;
2. accesso o registrazione utente;
3. visualizzazione menu principale;
4. selezione del livello;
5. svolgimento del quiz;
6. conclusione del livello;
7. aggiornamento progressi;
8. consultazione statistiche.

Questo flusso consente all'utente di muoversi in modo intuitivo nel percorso didattico.

Dettaglio di sequenza del quiz



Design della valutazione dei quiz

La valutazione delle risposte varia in base alla tipologia di domanda.

Domande a scelta multipla

La risposta viene verificata confrontando la soluzione proposta con quella corretta, passando attraverso il motore Prolog per validazioni sintattiche.

Domande aperte / fill-in-the-blank

La risposta viene inserita dall'utente e controllata sia dal punto di vista sintattico sia logico tramite il motore Prolog.

Gestione dell'errore

In caso di risposta non valida o eccezione interna:

- l'esito viene considerato negativo;
- l'applicazione continua a funzionare senza interrompere il flusso.

Gestione dei progressi utente

Il sistema mantiene traccia dei progressi per ogni utente autenticato.

Dati registrati

Per ogni livello completato vengono salvati:

- tempo impiegato;
- numero di tentativi;
- numero di risposte corrette;
- numero di risposte totali;
- timestamp della sessione.

KPI calcolati

Dal modello vengono derivati indicatori utili alla visualizzazione delle statistiche:

- livelli completati;
- percentuale di accuratezza;
- tempo complessivo di utilizzo;
- quiz tentati.

Gestione degli errori

Il progetto adotta un approccio prudente e controllato nella gestione degli errori.

Scelte progettuali

- uso di `Either` nei servizi;
- validazione preventiva degli input;
- fallback sicuri nei repository;
- controllo dello stato della sessione prima delle operazioni.

Obiettivo

Evitare crash imprevisti e mantenere l'applicazione stabile anche in presenza di dati incoerenti o input non validi.

Scelte tecnologiche rilevanti

Le principali scelte adottate nel design di dettaglio sono:

- **Scala 3.8.1**: per il supporto a `given`, `using` e sintassi moderna;
- **ScalaFX**: per l'interfaccia grafica desktop;
- **uPickle**: per serializzazione e deserializzazione JSON;
- **tuProlog**: per la valutazione delle query Prolog;
- **SBT**: per build e dipendenze;
- **Java 21**: come runtime di esecuzione.

Considerazioni finali

Il design di dettaglio è stato pensato per garantire:

- modularità;
- estendibilità;
- chiarezza dei flussi;
- semplicità di manutenzione;
- coerenza tra parte didattica, logica applicativa e interfaccia utente.

L'architettura ottenuta consente di aggiungere nuovi livelli, nuove domande e ulteriori funzionalità senza dover stravolgere le componenti già esistenti.

Implementazione →

Implementazione

Gianfranco Branco

Model

Il mio contributo nel package `model` si è focalizzato sulla creazione di un dominio solido e sulla gestione della persistenza, garantendo una netta separazione tra la modellazione dei dati e i dettagli di archiviazione.

User

Ho implementato la `case class User` come entità centrale per l'anagrafica del sistema. L'utilizzo di una `case class` garantisce l'immutabilità dei dati, garantendo che i dati dell'utente non subiscano modifiche accidentali durante l'esecuzione, rendendo il flusso dei dati tra Controller e View estremamente prevedibile.

LevelRecord

Questa classe modella il singolo evento di completamento di un livello. Ogni volta che un utente termina un livello, vengono registrati diversi KPI insieme al timestamp della sessione. La struttura permette la coesistenza di più occorrenze per lo stesso livello, abilitando così una cronostoria dettagliata dei suoi progressi. Ho scelto di definire `LevelRecord` all'interno dello stesso file sorgente di `UserProgress` poiché le due entità sono strettamente interdipendenti.

UserProgress

Rappresenta l'aggregatore globale dei progressi di un utente, mettendo in relazione l'identità dell'utente con la sua cronologia di LevelRecord.

Ho implementato diversi metodi di utility all'interno della classe per calcolare i KPI globali in tempo reale, partendo dai dati grezzi:

```
    // Ore totali di utilizzo (approssimate)
    def totalHours: Double = history.map(_.timeSpentMinutes).sum / 60.0

  // Somma totale delle risposte corrette ottenute
  def totalCorrects: Int = history.map(_.quizCorrects).sum

  // Esempio di calcolo derivato: percentuale globale di successo
  def globalAccuracy: Double =
    if (totalAttempts > 0) (totalCorrects.toDouble / totalAttempts) * 100
    else 0.0
```

Questo approccio segue il pattern *Rich Domain Model*, dove la logica di calcolo è incapsulata direttamente nel Model. Questo semplifica notevolmente i Controller, che non devono conoscere i dettagli dei calcoli statistici ma si limitano a interrogare il modello per visualizzare i risultati.

UserSession

Questa classe agisce come un contenitore globale per l'utente autenticato. Permettendo di mantenere la persistenza in memoria della sessione attiva. Questo evita di dover passare l'oggetto `User` tra tutti i controller della View, centralizzando l'accesso alle informazioni dell'utente loggato e migliorando la manutenibilità del flusso di navigazione.

LevelSession

La sessione è gestita tramite una macchina a stati definita da `sealed trait LevelSessionState`. Questo permette di distinguere nettamente tra una sessione Idle (nessun livello in corso) e una sessione ActiveLevel, partita attiva con timestamp e contatori.

All'interno del `object LevelSession`, ho utilizzato il pattern *Matching* per garantire che le operazioni sui KPI vengano eseguite solo se esiste effettivamente una sessione attiva, evitando errori di stato inconsistente.

```
  // Registro una risposta
def addAttempt(isCorrect: Boolean): Unit =
  state match
    case currentState: ActiveLevel =>
      state = currentState.copy(
        quizAttempts = currentState.quizAttempts + 1,
        quizCorrects =
          currentState.quizCorrects + (if isCorrect then 1 else 0)
      )
    case Idle => ()
```

Questa classe separa i progressi "vivi" (temporanei) da quelli definitivi salvati su disco. Solo quando viene chiamato il metodo endLevel(), lo stato viene resettato a Idle e i dati finali vengono passati al sistema di persistenza per essere trasformati in un LevelRecord.

```
// Alla conclusione del livello restituisco i dati finali
def endLevel(): Option[ActiveLevel] =
  val finalState = state match
    case currentState: ActiveLevel => Some(currentState)
    case Idle                      => None
  state = Idle
  finalState
```

Codec

Per rendere persistenti gli `User` e gli `UserProgress`, ho implementato i relativi Codec nel package `model.codec`.

Ho fatto uso del *Single Responsibility Principle* poiché ho isolato i Codec, evitando di "sporcare" le classi di dominio con dettagli implementativi legati al formato JSON. In questo modo, se in futuro si decidesse di cambiare formato di salvataggio, basterebbe modificare il Codec senza toccare la struttura della classe `User`.

Per le classi `UserCodec` e `UserProgressCodec`, ho utilizzato la generazione automatica dei Codec tramite `macroRW`. Questo permette al compilatore di ispezionare la struttura delle classi e generare la logica di parsing a tempo di compilazione, garantendo performance elevate e riducendo gli errori manuali. Poiché le macro non sanno gestire nativamente tipi come `LocalDateTime` o `UUID`, ho fornito delle istanze `given` personalizzate. Tramite il metodo `.bimap`, ho definito esplicitamente come mappare questi tipi su stringhe JSON, permettendo alle macro di "comporre" i Codec più complessi.

```
// Esempio di mapping bidirezionale per un Value Type (User.Id)
implicit val idRW: ReadWriter[User.Id] =
  readwriter[String].bimap(
    id => id.toString,                        // Da oggetto a Stringa JSON
    s  => User.Id(UUID.fromString(s))          // Da Stringa JSON a oggetto
  )

// Derivazione automatica per l'entità principale
implicit val userRW: ReadWriter[User] = macroRW
```

Controller

Nella realizzazione dei controller, l'obiettivo primario è stato il disaccoppiamento della logica di controllo dai servizi di persistenza e dalla gestione dei dati.

Per implementare una #Dependency Injection# nativa e pulita, ho sfruttato le potenzialità di Scala 3, come l'utilizzo dei `given` e i parametri contestuali `using` per fornire i repository ai controller. In questo modo i controller non istanziano direttamente i servizi, ma li ricevono implicitamente dal contesto. Questo facilita lo scambio delle implementazioni senza modificare la logica interna del controller.

Ho curato personalmente lo sviluppo dei seguenti componenti:

RegistrationController

Gestisce il flusso di creazione di un nuovo account. Si occupa di validare i dati di input e di interagire con lo `UserRepository` per persistere le nuove credenziali, assicurando che non vi siano duplicati nel sistema.

EditUserController

Permette all'utente di modificare le proprie informazioni di profilo. Questo controller interagisce direttamente con la `UserSession` per identificare l'utente attivo e aggiornare i dati sia in memoria che su disco tramite i relativi codec.

LevelsController

Il `LevelsController` è stato realizzato in collaborazione con il mio collega, adottando un approccio che separa la gestione del flusso applicativo dall'analisi statistica dei dati di gioco.

Per quanto mi riguarda, mi sono occupato della progettazione della struttura portante del controller e della gestione del ciclo di vita dei contenuti, implementando il disaccoppiamento dai servizi di persistenza utilizzando le `given` di Scala 3 per iniettare `LevelRepository` e `UserProgressRepository`. Questo garantisce che il controller non dipenda da una specifica implementazione del file system.

Service

Il layer dei servizi funge da intermediario tra i Controller e la persistenza, garantendo che le regole di business siano centralizzate e indipendenti dall'interfaccia utente.

Repository

Ho implementato il pattern *Repository* per gestire l'accesso ai dati su file system. Ogni entità `User` , `Level` , `UserProgress` dispone di una *trait* e di un'implementazione concreta `Impl` che utilizza il formato JSON. In `UserProgressRepositoryImpl` , ho implementato una logica di aggiornamento "read-modify-write" che garantisce l'integrità dei dati: il sistema carica lo stato attuale, aggiorna solo il record interessato e sovrascrive il file atomicamente. Le ricerche all'interno dei file, come in `findByUserId` sono effettuate in modo funzionale tramite predicati sulle collezioni Scala, garantendo velocità e leggibilità.

UserService

Il `UserService` si occupa di validare le operazioni sugli utenti prima di persisterle. Nel metodo `signUp` , ho implementato controlli per impedire la creazione di utenti con nomi vuoti o duplicati, restituendo un tipo `Either[String, User]` per gestire gli errori in modo esplicito e senza eccezioni.

PrologGrader

In collaborazione con il mio collega, ho sviluppato il motore di valutazione delle risposte, che integra la libreria **tuProlog** all'interno dell'ecosistema Scala.

Il mio contributo è stato nei quiz di tipo Fill-in-the-blank. La logica non si limita a un confronto testuale, ma verifica la correttezza sintattica del codice inserito dall'utente istanziando una `Theory` Prolog dinamica. Il metodo `validate` fonde la conoscenza base del livello con la soluzione proposta dall'utente e tenta di risolvere una `testQuery` . Il successo della query determina la correttezza della risposta.

```
// Esempio di validazione dinamica Prolog
def validate(levelKnowledge: String, userSolution: String, testQuery: String): Boolean =
  try
    val prologEngine = new Prolog()
    val runtimeTheory = new Theory(levelKnowledge + "\n" + userSolution)
    prologEngine.setTheory(runtimeTheory)
    prologEngine.solve(testQuery).isSuccess // La correttezza è determinata dal successo logico
  catch
    case _: Exception => false
```

View

L'interfaccia grafica è stata sviluppata utilizzando **ScalaFX**, un wrapper Type-Safe di JavaFX che permette di sfruttare appieno la sintassi dichiarativa di Scala 3. L'obiettivo è stato creare un'esperienza utente coerente, minimizzando la duplicazione del codice tramite l'astrazione dei componenti.

RegistrationPage

In questa schermata di registrazione utente, ottengo l'input dell'username e lo passo al `RegistrationController`. La vista gestisce i feedback di errore tramite callback che aggiornano dinamicamente l'interfaccia senza ricaricare l'intera scena.

EditUserPage

Questa vista permette la gestione della modifica od eliminazione dell'utenza. Prima di procedere all'eliminazione dell'utente richiamo la modale `showCustomConfirm` e una volta confermata l'azione, la vista coordina con il controller il logout e il reindirizzamento alla pagina di registrazione.

LevelsPage

In collaborazione con il team, ho curato la struttura a griglia dei livelli. Ho implementato il layout principale utilizzando il componente `cardGrid` e la logica di navigazione che permette di selezionare un livello e passare alla fase di gioco.

UIComponents

In collaborazione con il mio collega, ho contribuito allo sviluppo dell'oggetto `UIComponents`. Questo componente funge da "libreria di design" interna, centralizzando lo stile e i layout comuni per garantire uniformità visiva in tutta l'applicazione.

Mi sono occupato del metodo `showCustomConfirm` per permettere la creazione di modali custom. A differenza dei dialoghi standard di sistema, questa soluzione utilizza uno `Stage` con `StageStyle.Undecorated`, permettendo un controllo totale sul design che resta coerente a tutta l'applicazione, non utilizzando la modale standard del SO.

La modale accetta una funzione `onConfirm: () => Unit` come parametro. Questo approccio permette di slegare il componente grafico dalla logica applicativa: la modale si occupa solo della visualizzazione, delegando l'azione al chiamante.

Infine mi sono occupato della funzione `backButton`, per permettere all'utente il ritorno alla pagina precedente.

```
// Esempio della logica di gestione della modale da me implementata
def showCustomConfirm(head: String, message: String, onConfirm: () => Unit): Unit = {
  val dialog = new Stage {
    initModality(Modality.ApplicationModal)
    initStyle(StageStyle.Undecorated)
    // ... setup del layout e degli stili CSS-inline ...
    children = Seq(
      styledButton(confirmButtonMsg, "#d32f2f", "white", {
        onConfirm() // Esecuzione della logica di business
        delegate.getScene.getWindow.hide()
      })
    )
  }
  dialog.showAndWait()
}
```

Francesco Agnoletti

Model

Question

Ho lavorato sull'evoluzione del modello `Question`, estendendolo per supportare in modo più flessibile le diverse tipologie di domanda presenti nel progetto. In particolare, la struttura della domanda è stata resa più robusta per gestire sia la validazione della risposta sia i casi in cui sia necessario un controllo più articolato tramite Prolog.

```
def isCorrect(answer: String, theory: String = ""): Boolean =
    qType match
        case QuestionType.MultipleChoice =>
            PrologGrader.validateMultipleChoice(theory, answer, correctAnswer)
        case QuestionType.OpenQuestion =>
            validationQuery match
                case Some(query) => PrologGrader.validate(theory, answer, query)
                case None        => answer.trim == correctAnswer.trim
```

QuestionType

Ho introdotto e utilizzato il tipo `QuestionType` per distinguere chiaramente tra domande a risposta multipla e domande aperte. Questo ha permesso di rendere il flusso del quiz più chiaro, separando in modo esplicito le diverse logiche di visualizzazione e validazione.

Controller

LoginController

Ho contribuito alla gestione del login utente e della navigazione iniziale dell'applicazione. Il controller si occupa di validare l'input, recuperare l'utente autenticato e indirizzare correttamente l'utente verso il menu principale o verso la registrazione.

MenuController

Il menu principale rappresenta il punto di accesso alle varie funzionalità dell'applicazione. Questo controller coordina la navigazione verso le schermate principali, mantenendo separata la logica di controllo dalla presentazione grafica.

QuizController

Mi sono occupato della gestione del flusso del quiz, in particolare dell'avanzamento tra le domande, della validazione della risposta e del passaggio automatico alla schermata successiva. Ho lavorato anche sulla chiusura del livello e sul recupero dei dati necessari per mostrare il riepilogo finale, mantenendo centralizzata la logica applicativa nel controller.

StatsController

Ho collaborato alla creazione del controller dedicato alle statistiche dell'utente corrente, così da recuperare in modo centralizzato i dati relativi ai progressi e ai risultati ottenuti nei livelli completati. Questo ha permesso di esporre in modo ordinato le informazioni utili alla schermata delle statistiche, senza distribuire la logica di calcolo nelle View.

Services

PrologGrader

Mi sono occupato dello spostamento e della sistemazione di `PrologGrader` nel package dei servizi, così da collocare correttamente la logica di valutazione nel livello applicativo più adatto. Questo intervento ha reso più chiara la separazione tra dominio, servizi e interfaccia grafica.

Ho inoltre corretto alcuni aspetti legati all'interpretazione delle regole Prolog, in modo da rendere più affidabile la validazione delle risposte. Questo lavoro è stato fondamentale per gestire correttamente i quiz basati su Prolog, soprattutto nelle domande a risposta multipla.

```
def validateMultipleChoice(
    levelKnowledge: String,
    userAnswer: String,
    correctAnswer: String
    ): Boolean =
    try
        val normalizedUser = userAnswer.trim
        val normalizedCorrect = correctAnswer.trim

        // Verifica che entrambe siano codice Prolog corretto
        val userTheory = new Theory(levelKnowledge + "\n" + normalizedUser)
        val correctTheory = new Theory(levelKnowledge + "\n" + normalizedCorrect)

        normalizedUser == normalizedCorrect
    catch
        case _: Exception => false
```

LevelRepository

Ho contribuito anche alla parte di caricamento dei livelli, intervenendo sulla gestione del repository in modo da rendere più robusta la lettura del file `levels.json`. Questo ha permesso all'applicazione di reperire correttamente i dati dei livelli sia dalle risorse interne sia dal percorso di esecuzione locale, facilitando la fase di deployment ed il popolamento dei livelli.

```
private def loadFromResource(): Option[List[Level]] =

  try
    Option(getClass.getResourceAsStream("/levels.json")).flatMap { stream =>
      try
        val content = Source.fromInputStream(stream).mkString
        if content.isBlank then Some(List.empty)
        else
          try Some(read[List[Level]](content))
```

```
            catch case _ => Some(List.empty)
        finally stream.close()
      }
    catch
      case _ => None
```

View

SplashView

Ho lavorato sulla schermata iniziale dell’applicazione, curando la splash screen mostrata all’avvio. L’obiettivo era offrire un primo impatto semplice e pulito, coerente con l’identità visiva del progetto.

UIComponents

Ho contribuito allo sviluppo dei componenti grafici riutilizzabili, così da uniformare lo stile dell’applicazione e ridurre la duplicazione di codice. In particolare, questa parte ha permesso di centralizzare elementi comuni come pulsanti stilizzati, layout di base, logo e finestre di conferma personalizzate.

QuizMultiplePage

Ho lavorato alla schermata del quiz a risposta multipla, curando la disposizione degli elementi e la gestione delle interazioni dell’utente. L’obiettivo era mantenere un’interfaccia chiara e coerente con il resto dell’applicazione, rendendo la selezione della risposta semplice e immediata.

StatsPage

Ho contribuito anche alla schermata delle statistiche dell’utente, che mostra in modo ordinato i progressi accumulati durante l’utilizzo dell’applicazione. Questa vista consente di visualizzare i dati in una forma più leggibile e rappresenta un completamento naturale del percorso di gioco.

Ho lavorato nel complesso sulla coerenza del flusso tra le varie schermate del quiz, così da rendere il passaggio tra domande, feedback e riepilogo finale più fluido e intuitivo per l’utente.

Testing →

Testing

La strategia di testing adottata si è focalizzata sulla robustezza del **Model** e della **Business Logic**, garantendo che il cuore applicativo (motore Prolog e gestione sessioni) sia resiliente a stati inconsistenti o input errati.

Tecnologie e Metodologia

- **Framework**: Abbiamo utilizzato **ScalaTest** adottando la suite **AnyFunSuite** per la sua chiarezza nell'esecuzione dei test.
- **Lifecycle Management**: Grazie al trait **BeforeAndAfterEach**, abbiamo garantito l'isolamento dei test resettando lo stato dei componenti singleton, come `LevelSession`, prima di ogni esecuzione.

Coverage e Grado di Copertura

L'analisi della copertura del codice è stata effettuata tramite lo strumento **Scoverage**:

All packages	70.58%
prologfordummies.model.codec	98.51%
prologfordummies.model	76.19%
prologfordummies.services	70.33%
prologfordummies.services.prolog	36.00%

SCoverage generated at Sun Mar 29 16:07:50 CEST 2026

Lines of code:	2239	Files:	33	Classes:	41	Methods:	145
Lines per file:	67.85	Packages:	7	Classes per package:	5.86	Methods per class:	3.54
Total statements:	1523	Invoked statements:	283	Total branches:	107	Invoked branches:	44
Ignored statements:	0						
Statement coverage:	70.58 %			Branch coverage:	41.12 %		

Class	Source file	Lines	Methods	Statements	Invoked	Coverage		Branches	Invok
Id	User.scala	25	2	6	6		100.00 %	0	0
Level	Level.scala	14	1	2	2		100.00 %	0	0
Name	User.scala	30	2	2	2		100.00 %	0	0
QuestionType	QuestionType.scala	13	3	6	6		100.00 %	0	0

Analisi della Copertura e Strategia di Testing

L'attività di testing è stata focalizzata sul **kernel logico** dell'applicazione, garantendo stabilità dove il rischio di bug critici è maggiore.

Risultati di Copertura (Scoverage)

I dati estratti mostrano un'eccellente affidabilità dei componenti core:

- `model.codec` **(98.51%)**: Copertura pressoché totale. Garantisce l'integrità della serializzazione JSON e della persistenza dei dati.
- `model` **(76.19%)**: Elevata robustezza nella gestione degli stati e delle entità di dominio.
- `services` **(70.33%)**: Solida validazione dei repository e della business logic di gestione utente.
- `services.prolog` **(36.00%)**: Copertura focalizzata sui flussi principali di validazione.

Da questi dati sono esclusi i componenti puramente visuali e di coordinamento (View e Controller), i quali sono stati testati manualmente.

Esempi Rilevanti di Testing

1. Gestione della Sessione di Gioco (LevelSession)

Il test verifica che la `LevelSession` gestisca correttamente i contatori dei tentativi e che resetti lo stato dopo la chiusura di un livello.

```
test("addAttempt dovrebbe aggiornare i contatori dei Quiz tentati e corretti") {
  val levelId = Level.Id.random
  LevelSession.startLevel(levelId)
  LevelSession.addAttempt(isCorrect = true)
  LevelSession.addAttempt(isCorrect = false)
  LevelSession.addAttempt(isCorrect = true)

  val stats = LevelSession.currentStats.getOrElse((0, 0))
  assert(stats._1 == 3) // 3 tentativi totali
  assert(stats._2 == 2) // 2 risposte corrette
}

test("endLevel dovrebbe restituire i dati finali e tornare allo stato Idle") {
  LevelSession.startLevel(Level.Id.random)
  LevelSession.addAttempt(isCorrect = true)

  val finalData = LevelSession.endLevel().getOrElse(fail("Dati mancanti"))
  assert(finalData.quizCorrects == 1)
  assert(LevelSession.currentStats.isEmpty) // Stato resettato correttamente
}
```

2. Test per la valutazione di domande aperte con validazione Prolog

Il test verifica che la logica inserita dall'utente sia semanticamente corretta. A differenza di un semplice confronto testuale, il sistema utilizza il motore **tuProlog** per verificare se il codice dell'utente soddisfa il `validationQuery` (Goal) impostato per il quiz.

```
test("Valutazione logica Prolog - caso positivo") {
  val staticTheory = "parent(abramo, isacco). parent(isacco, giacobbe)."
  val userCode = "nonno(X, Y) :- parent(X, Z), parent(Z, Y)."
  val goal = "nonno(abramo, giacobbe)."

  val q = Question(
    id = 1,
    question = "Definisci la regola nonno",
    correctAnswer = "",
```

```
      answers = List.empty,
      qType = QuestionType.OpenQuestion,
      validationQuery = Some(goal)
    )

    assert(q.isCorrect(userCode, staticTheory))
  }
```

3. Test delle domande a risposta multipla

È stata realizzata anche una piccola suite di test dedicata al modello `Question` per verificare il comportamento delle domande a risposta multipla.
In particolare, i test controllano che la domanda venga inizializzata correttamente e che il metodo `isCorrect` restituisca l'esito atteso sia nel caso di risposta corretta sia in quello di risposta errata.

```
test("La lista delle risposte deve contenere la risposta corretta (MultipleChoice)") {
    val question = Question(
      id = 1,
      question = "Qual è la capitale d'Italia?",
      correctAnswer = "Roma",
      answers = List("Roma", "Milano", "Napoli"),
      qType = QuestionType.MultipleChoice
    )

    assert(question.answers.contains(question.correctAnswer))
  }

test("isCorrect dovrebbe restituire falso per la risposta sbagliata (MultipleChoice)") {
    val question = Question(
      id = 1,
      question = "Qual è la capitale della Francia?",
      correctAnswer = "Parigi",
      answers = List("Roma", "Berlino", "Parigi"),
      qType = QuestionType.MultipleChoice
    )

    assert(!question.isCorrect("Roma"))
  }
```

Retrospettiva →

Kickoff Meeting (31/01/2026)

Punti chiave del primo incontro

Il kickoff meeting ha segnato l’avvio ufficiale delle attività di progetto. L’incontro ha avuto l’obiettivo di creare una visione condivisa all’interno del team, ponendo particolare attenzione a tre aspetti fondamentali.

Allineamento

Comprensione comune degli obiettivi progettuali e delle aspettative del cliente, così da garantire una direzione chiara e condivisa fin dalle fasi iniziali.

Perimetro

Definizione precisa dell’ambito del progetto, individuando ciò che rientra nelle attività previste e stabilendo i confini funzionali del prodotto.

Ruoli

Assegnazione delle responsabilità ai membri del team, chiarendo compiti e contributi attesi da ciascun partecipante.

Product Backlog

Sulla base delle informazioni emerse durante il kickoff meeting, le necessità del prodotto sono state raccolte e organizzate nel **Product Backlog**, che rappresenta il riferimento principale per la pianificazione, la gestione e il monitoraggio dello sviluppo futuro.

Retrospettiva

Il progetto realizzato soddisfa i requisiti minimi previsti inizialmente e include un'introduzione alle funzionalità opzionali immaginate in fase di analisi. Nel complesso, il risultato finale è coerente con gli obiettivi prefissati e con la struttura pensata durante la progettazione.

Un possibile miglioramento futuro riguarderebbe l'implementazione delle funzionalità opzionali definite in fase di analisi ed un ulteriore raffinamento dell'interfaccia grafica.

Lo sviluppo si è svolto seguendo un approccio Agile, con una pianificazione per sprint e una verifica costante delle funzionalità implementate. Questo ha permesso di produrre incrementi concreti e facilmente testabili, mantenendo sempre un legame diretto tra sviluppo e utilizzo finale.

L’organizzazione del lavoro è stata generalmente rispettata, consentendo di portare avanti le attività con continuità e di completare la maggior parte degli obiettivi nei tempi stabiliti.

Kickoff →

Sprint Planning 1

Periodo: 01/02/2026 – 11/02/2026

Backlog:

ID	Priorità	Task	ISE
1	Alta	Modellazione utente	3
2	Alta	Autenticazione utente	2
3	Alta	Registrazione utente	1
4	Alta	Eliminazione utente	1
5	Alta	Modifica utente	1
6	Alta	Interfaccia grafica utente (Registrazione)	4
7	Alta	Interfaccia grafica utente (Login)	4
8	Alta	Interfaccia grafica utente (Modifica ed eliminazione)	4
9	Alta	Sistema persistenza utenti	4
10	Alta	Modellazione Quiz a risposta multipla	3
11	Alta	Modellazione Quiz "Fill-in-the-blank"	3
12	Alta	Interfaccia grafica interazione Utente-Quiz	6
13	Media	Valutazione risposte	8
14	Media	Sistema persistenza Quiz	4
15	Media	Modellazione Level	3
16	Media	Sistema persistenza Level	4
17	Media	Interfaccia grafica percorso didattico	4
18	Media	Sistema persistenza progressi Utente	6
19	Media	Testing logica applicativa	8
20	Media	Dashboard progressi utente	5
21	Media	Contenuti teorici livelli	5
22	Media	Contenuti Quiz Domande e Risposte	5

ID	Priorità	Task	ISE
23	Bassa	Rifinitura UX	5
24	Bassa	Modalità Survival	15
25	Bassa	Playground Prolog	20

Sprint Goal

Configurazione iniziale del progetto, creazione dei Mockup, definizione della documentazione di avvio, pianificazione strategica (Backlog) e implementazione del nucleo funzionale dell'utente (Modello, CRUD e interfaccia di Login).

Deadline

La scadenza dello sprint è il **11/02/2026**.

Task completati nello Sprint

Project Management

Backlog & Planning — Analisi dei requisiti e ultimazione della tabella di backlog con priorità e stime.

Documentazione

Kickoff & Index — Stesura del documento di kickoff e creazione dell'indice strutturato per la documentazione tecnica.

Architettura

Struttura MVC — Definizione dei package e configurazione dell'architettura Model-View-Controller.

Brand Identity

Visual Design — Creazione del logo ufficiale "Prolog For Dummies" e integrazione degli asset grafici.

Sviluppo Core

User Model & CRUD — Modellazione dell'entità User e implementazione della logica di gestione dati (Create, Read, Update, Delete).

GUI

ScalaFX & Login — Configurazione del framework ScalaFX e sviluppo della prima versione della LoginPage.

Quality Assurance

Unit Testing — Creazione della suite di test per validare il modello e le operazioni CRUD dell’utente.

Sprint Review

Pianificazione

La redazione del backlog fornisce una visione chiara dei task prioritari e una stima realistica per i futuri cicli di sviluppo.

Architettura

Imbastita la struttura MVC per il progetto e a specchio anche nella sezione Test.

User Logic

Il modello utente è stabile, testato tramite unit test e pronto per l’integrazione con sistemi di persistenza.

Interfaccia

La LoginPage è funzionale e coerente con l’identità visiva.

Sprint Retrospective

Cosa è andato bene

- Organizzazione — Il backlog definito subito ha ridotto l’incertezza decisionale.
- Approccio TDD — I test implementati insieme al modello User hanno aumentato l’affidabilità del core.
- Documentazione — Kickoff e indice creati all’inizio hanno migliorato tracciabilità e chiarezza.

Difficoltà incontrate

- Setup ScalaFX — Tempo necessario per configurare correttamente le dipendenze in Scala 3.
- Prioritarizzazione — La stima iniziale dei task ha richiesto confronti per allineare la percezione della complessità.

Miglioramenti previsti per Sprint 2

- Estensione GUI — Implementare della pagian di Registraizione mantenendo coerenza stilistica con la Login.
- Persistenza — Evolvere il CRUD dell'utente per avere persistenza dei dati

- Navigazione — Implementare gestione dinamica del cambio scena (scene switching).

Sprint Planning 2

Periodo: 12/02/2026 – 22/02/2026

Sprint Goal

Completare l’interfaccia di registrazione, introdurre la persistenza locale per gli utenti e abilitare il cambio scene dinamico. Preparare il terreno per l’inserimento dei primi quiz.

Deadline

La scadenza dello sprint è il **22/02/2026**.

Task completati nello Sprint

Interfaccia

Pagina Registrazione — sviluppo e integrazione della schermata di registrazione con le stesse regole stilistiche della login.

Scene switching — meccanismo funzionale per passare da una scena all'altra (login ➜ registrazione ➜ dashboard).

Persistenza

Storage utenti — introduzione di un file/DB locale per salvare gli oggetti `User` creati e letti dal CRUD.

Logica utente

Validazione — controllo dei campi durante la registrazione e modifica con messaggi di errore visibili.

Quality Assurance

Test persistenti — estensione della suite di unit test per verificare le operazioni CRUD atte alla persistenza.

Sprint Review

Progresso

Le funzionalità di base dell'utente ora conservano i dati tra le sessioni e l'esperienza GUI è navigabile.

Architettura

Nessuna modifica sostanziale, ma il package `persistence` è stato aggiunto con le classi di salvataggio.

Utente

Registrazione e login sono operativi con validazione e salvataggio; resta da sviluppare la pagina di modifica/eliminazione.

Qualità

I test sono passati e l'infrastruttura per testare la persistenza è stabile.

Sprint Retrospective

Cosa è andato bene

- Persistenza semplice e veloce da introdurre grazie alla struttura MVC chiara.
- Scene switching è risultato molto utile per prototipare percorsi utente.

Difficoltà incontrate

- Gestione degli errori di I/O (file corrotto, permessi) ha richiesto più tempo del previsto.
- La validazione lato client ha rivelato casi limite non previsti inizialmente.

Miglioramenti previsti per Sprint 3

- Completare CRUD utente (modifica/eliminazione interface e storage).
- Iniziare la modellazione dei quiz (risposta multipla e fill-in-the-blank).
- Progettare interfaccia grafica per l'interazione Utente-Quiz.

Sprint Planning 3

Periodo: 23/02/2026 – 04/03/2026

Sprint Goal

Completare le funzionalità CRUD, introdurre la gestione della sessione utente ed estendere il sistema di persistenza ai livelli di gioco, intervenendo sull’architettura applicativa per garantire una piena conformità al pattern MVC e rafforzare il disaccoppiamento tra View e Controller.

Deadline

La scadenza dello sprint è il **04/03/2026**.

Task pianificati per lo Sprint

Utente

Interfaccia modifica utente — sviluppo della schermata per modificare il nome utente, con validazione e messaggi di errore.

Interfaccia eliminazione utente — implementazione della funzionalità per eliminare l'account utente, con conferma e pulizia dei dati.

Visualizzazione livelli — interfaccia per mostrare i livelli disponibili all'interno dell'applicazione.

Persistenza

Aggiornamento storage — estensione del sistema di persistenza per supportare modifica ed eliminazione degli utenti e dei livelli.

Quality Assurance

Test CRUD completo — estensione dei test per coprire modifica ed eliminazione utenti.

Sprint Review

Progresso

Lo sprint ha completato con successo il CRUD utente, includendo interfaccia per modifica ed eliminazione, persistenza aggiornata e test estesi. Inoltre, durante lo sviluppo è emerso che, per gestire la navigazione tra più schermate(Statistiche, Modifica Utente) e mantenere lo stato dell'utente è necessaria una Sessione utente. L'autenticazione è stata perciò integrata con l'implementazione della gestione della sessione utente.

L'introduzione dell'interfaccia di visualizzazione dei livelli, ha posto le basi per lavorare sull'interazione utente-quiz. Infine, oltre all'implementazione delle nuove funzionalità, una parte significativa dello Sprint è stata dedicata al refactoring strutturale. È stata operata una migliore separazione tra la logica di business e la presentazione. Grazie alla migrazione di alcune logiche che erano presenti nelle View e che sono state inserite nei rispettivi Controller.

Architettura

Nessuna modifica sostanziale all'architettura esistente. La persistenza dei dati è stata estesa per supportare le nuove operazioni CRUD e l'introduzione dei livelli.

Utente

Il CRUD utente è ora completo: registrazione, login, modifica ed eliminazione sono operativi con validazione e salvataggio.

Qualità

Tutti i test sono passati, inclusi quelli per le nuove operazioni CRUD. L'infrastruttura di testing è robusta e copre i casi limite. La rifattorizzazione del codice ha migliorato la separazione dei componenti del progetto, rendendolo più chiaro e manutenibile.

Sprint Retrospective

Cosa è andato bene

- L'estensione del CRUD è stata integrata senza problemi grazie alla struttura MVC consolidata.
- La persistenza si è dimostrata flessibile per aggiungere operazioni di modifica ed eliminazione.
- L'utilizzo di branch specifici per suddividere i diversi lavori si è dimostrato efficace e non ha generato problemi durante le operazioni di merge.

Difficoltà incontrate

- Gestione della conferma per l'eliminazione utente ha richiesto attenzione per evitare cancellazioni accidentali.
- Validazione dei campi in modifica ha rivelato la necessità di controlli più rigorosi sui dati esistenti.

Miglioramenti previsti per Sprint 4

- Iniziare l'implementazione dei quiz con modellazione delle domande e risposte (risposta multipla e fill-in-the-blank).
- Implementare la valutazione automatica delle risposte e feedback immediato.
- Progettare l'interfaccia grafica per l'interazione Utente-Quiz.
- Consolidare il percorso didattico strutturato per livelli.
- Aggiungere contenuti teorici (Theory) per ciascun livello.
- Completare la rimozione della logica applicativa residua dalle View, trasferendola nei Controller.
- Migliorare la GUI rendendola più responsiva.

Sprint Planning 4

Periodo: 05/03/2026 – 15/03/2026

Sprint Goal

Implementare il sistema completo dei quiz, estendere il modello delle domande per supportare tipologie diverse (scelta multipla e risposta aperta), introdurre la nuova interfaccia dedicata alle domande aperte e rifattorizzare l'architettura per garantire pieno rispetto del pattern MVC. Completare inoltre l'integrazione dei contenuti teorici e strutturare correttamente il percorso didattico basato sui livelli.

Deadline

La scadenza dello Sprint è il **15/03/2026**.

Task pianificati per lo Sprint

Quiz Engine

Introduzione del campo qType nel modello Question — aggiunta della proprietà che identifica la tipologia di domanda (MultipleChoice o OpenQuestion).

Creazione delle domande aperte — estensione del dataset e aggiunta di un nuovo livello dedicato esclusivamente a domande open.

Routing automatico delle viste — implementazione della logica nel Controller per selezionare dinamicamente la View corretta (QuizPage o OpenQuizPage) in base al tipo della domanda.

Feedback utente per la risposta — integrazione di un feedback all'utente dopo la risposta.

Interfaccia Grafica

Implementazione OpenQuizPage — nuova schermata dedicata alle domande con risposta aperta, dotata di TextArea e pulsante di conferma.

Aggiornamento QuizPage — adeguamento della schermata delle domande a scelta multipla per supportare il nuovo flusso gestito dal Controller.

Architettura e Persistenza

Aggiornamento levels.json — conversione di tutte le domande esistenti in MultipleChoice e aggiunta del nuovo livello con domande OpenQuestion.

Refactoring MVC — rimozione della logica residua nelle View e centralizzazione della gestione del flusso (prossima domanda, validazione risposta, routing) nel QuizController.

Quality Assurance

Test delle nuove tipologie di domande — verifica del corretto funzionamento di domande a risposta multipla e aperta.

Sprint Review

Progresso

Lo Sprint ha introdotto il supporto completo ai quiz, permettendo la gestione di due tipologie di domande: multiple choice e open question. Il modello Question è stato esteso con il campo qType, e l'intero dataset è stato aggiornato di conseguenza, includendo un nuovo livello dedicato alle risposte aperte.

Il Controller ora gestisce interamente il flusso dei quiz: la validazione della risposta, il feedback visivo, il delay e il passaggio alla prossima domanda. La View è diventata completamente passiva e aderente ai principi MVC.

L'implementazione della nuova interfaccia OpenQuizPage ha ampliato notevolmente il potenziale del sistema didattico, consentendo domande più articolate e un'interazione più naturale da parte dell'utente.

Infine, un deciso intervento di refactoring ha permesso di applicare il principio DRY, centralizzando la creazione di elementi comuni (come i pulsanti di navigazione "Indietro") per eliminare ridondanze e migliorare la coerenza estetica dell'applicazione.

Architettura

Le principali modifiche architetturali riguardano:

- Il QuizController ora è l'unica fonte di logica applicativa per i quiz.
- Le View sono state semplificate e rese stateless.
- Introdotto un routing dinamico basato su qType per la selezione della pagina corretta.
- Esteso il sistema di persistenza dei livelli con nuovi campi e nuove tipologie di domande.
- Implementazione di showCustomConfirm per gestire avvisi e conferme tramite callback, separando la logica di business (cosa succede al click) dalla logica di visualizzazione (come appare la finestra)

Utente

L'utente ora può:

- Rispondere a domande a scelta multipla con feedback immediato.
- Interagire con domande aperte tramite una TextArea dedicata.
- Navigare fluidamente tra le domande.

L’esperienza utente è risultata più fluida, chiara e coerente grazie alla struttura unificata del flusso dei quiz.

Qualità

I test effettuati hanno confermato:

- Corretta visualizzazione di ogni tipo di domanda.
- Routing affidabile e privo di regressioni.
- Nessuna interferenza tra i due tipi di quiz.

L’infrastruttura di testing risulta adeguata e pronta per future estensioni (punteggio, risultati, tracking risposte).

Sprint Retrospective

Cosa è andato bene

- L’estensione del modello Question è stata integrata senza modificare il codice esistente.
- Il refactoring del Controller ha migliorato drasticamente la separazione delle responsabilità.
- Le Views risultano ora semplici, chiare e facilmente estendibili.
- La nuova OpenQuizPage ha ampliato le capacità didattiche dell’app.
- L'aggiornamento dei livelli presenti nel dataset è avvenuta senza problemi.

Difficoltà incontrate

- Si sono verificati alcuni conflitti durante l’integrazione delle nuove funzionalità, che hanno richiesto tempo per essere risolti correttamente.
- L’aggiunta del campo qType ha richiesto una revisione accurata del modello, del dataset e del routing, comportando un refactoring più complesso del previsto.
- L’aggiornamento coordinato tra più file (model, controller, viste e JSON) ha richiesto test e verifiche aggiuntive per evitare regressioni.

—

Miglioramenti previsti per Sprint 5

- Introduzione di un sistema di punteggio basato sulle risposte corrette.
- Creazione della schermata finale con riepilogo del quiz.
- Aggiunta della schermata dei Progressi Utenti.
- Miglioramenti all’interfaccia per facilitare l’accessibilità e la responsività.
- Aggiunta della validazione delle risposte aperte con motore Prolog.
- Aggiunta della validazione delle risposte chiuse con motore Prolog.

Sprint Planning 5

Periodo: 16/03/2026 – 29/03/2026

Sprint Goal

Completare il ciclo di gioco introducendo il sistema di punteggio, la schermata finale di riepilogo del quiz e la persistenza dei progressi utente. Inoltre, estendere la validazione delle risposte tramite motore Prolog, migliorare la gestione della sessione di gioco e rafforzare la qualità complessiva dell'interfaccia e dell'architettura applicativa.

Deadline

La scadenza dello Sprint è il **29/03/2026**.

Task pianificati per lo Sprint

Quiz e Valutazione

Sistema di punteggio — introduzione del calcolo dei risultati in base alle risposte corrette e al numero totale di tentativi.

Schermata finale del quiz — sviluppo della vista conclusiva con riepilogo delle prestazioni dell'utente al termine del livello.

Validazione tramite Prolog — integrazione del motore Prolog per la validazione delle risposte, sia per le domande aperte sia per quelle a scelta multipla.

Progressi Utente

Persistenza dei progressi — salvataggio dei risultati del quiz associati all'utente autenticato.

Storico completamento livelli — registrazione dei tentativi e dei livelli completati per consentire future analisi dei progressi.

Architettura e Gestione Sessione

Gestione sessione di gioco — estensione della sessione livello per tracciare correttamente tentativi, risposte corrette e tempo trascorso.

Refactoring del controller quiz — consolidamento della logica di fine livello, avanzamento e salvataggio dei dati in un unico punto di controllo.

Interfaccia Grafica

Miglioramenti grafici — affinamento della UI finale e delle finestre di conferma per una migliore esperienza utente.

Coerenza visiva — uniformare i componenti grafici delle diverse schermate del quiz.

Quality Assurance

Test delle funzionalità di sessione e progressi — verifica del corretto salvataggio dei dati e della gestione delle statistiche di gioco.

Sprint Review

Progresso

Lo Sprint ha completato il flusso di gioco introducendo la fase finale del quiz e il salvataggio dei progressi dell'utente. Al termine del livello, l'applicazione ora raccoglie statistiche sul quiz svolto, calcola il riepilogo finale e mostra una finestra di conferma con i risultati ottenuti.

È stata inoltre introdotta la persistenza dei progressi utente, così da registrare in modo strutturato i livelli completati, il numero di tentativi, le risposte corrette e il tempo impiegato. Questo ha permesso di dare continuità all'esperienza di gioco e di preparare il terreno per la futura schermata dei progressi.

Sul fronte della logica applicativa, il controller dei quiz è stato ulteriormente rafforzato: ora gestisce non solo la selezione della prossima domanda, ma anche la chiusura del livello, il calcolo delle statistiche e il salvataggio dei risultati. La gestione della sessione di gioco è stata estesa per supportare il tracciamento completo delle prestazioni dell'utente.

Architettura

Le principali modifiche architetturali riguardano:

- Centralizzazione della logica di fine livello nel QuizController.
- Introduzione della persistenza dei progressi utente.
- Estensione della sessione di livello per raccogliere statistiche di gioco.
- Separazione più netta tra flusso applicativo, persistenza e presentazione.
- Miglioramento dell'architettura per la schermata dedicata ai progressi.

Utente

L'utente ora può:

- Completare un quiz e visualizzare un riepilogo finale.

- Ricevere un feedback più chiaro sui risultati ottenuti.
- Vedere tracciati i propri progressi in modo persistente.
- Visualizzare un riepilogo completo dei risultati del livello svolto.

Qualità

I test introdotti e quelli già presenti hanno confermato:

- Corretto avanzamento del flusso quiz.
- Persistenza affidabile dei progressi.
- Calcolo coerente delle statistiche finali.
- Nessuna regressione rispetto alle funzionalità introdotte negli sprint precedenti.

L'infrastruttura di testing risulta ora più solida e adatta a supportare ulteriori estensioni, come una schermata dedicata ai progressi complessivi e analisi più avanzate dei risultati.

Sprint Retrospective

Cosa è andato bene

- La chiusura del flusso quiz ha reso l'applicazione più completa e soddisfacente per l'utente.
- Il salvataggio dei progressi ha introdotto una base solida per la parte di tracking e monitoraggio.
- Il controller ha beneficiato di un ulteriore accentramento della logica, migliorando la leggibilità del codice.
- La gestione della sessione di livello si è dimostrata utile per mantenere statistiche coerenti durante tutta la partita.

Difficoltà incontrate

- La sincronizzazione tra sessione, persistenza e interfaccia finale ha richiesto attenzione per evitare inconsistenze nei dati salvati.
- La definizione del momento corretto in cui chiudere il livello e mostrare il riepilogo ha richiesto alcuni aggiustamenti logici.
- L'integrazione della persistenza dei progressi ha richiesto una revisione dell'architettura già consolidata negli sprint precedenti.

Conclusioni

Complessivamente il team è soddisfatto del lavoro effettuato nonostante le difficoltà riconducibili al nuovo paradigma di programmazione. Ogni componente è stato in grado di portare a termine i propri task, mirando a uno stile di programmazione il più funzionale possibile. Lo sviluppo dell'applicazione è avvenuto tramite un approccio Test-Driven Development, seppur non sempre sia stato rispettato in maniera rigorosa.